

Master-Projekt:

Smart Campus: Digitalisierung der Wasserzähler

ausgeführt von:

Krzysztof Struzyna

krzysztof.struzyna@student.hs-rm.de

Matr.-Nr.: 1159963

Betreuender Dozent: Prof. Dr. Heinz Werntges

26.9.2024

Inhaltsverzeichnis

1	Einleitung	1
1.1	Ziele des Projektes	1
1.2	Aufbau der Arbeit	2
2	Wasserzähler-Halterung	3
2.1	Konzept	4
2.2	Modellierung	6
2.3	3D-Druck	9
3	AI-on-the-edge	10
3.1	Über das Projekt	10
3.2	Inbetriebnahme	11
3.2.1	Flashen des ESP32-Cam	11
3.2.2	Anpassung der Brennweite für scharfe Nahaufnahmen . .	13
3.2.3	Initiale Konfiguration	14
3.3	Betrieb von „AI-on-the-edge“	15
3.4	Herausforderungen und Probleme	16
4	MeterMonitor	19
4.1	Ziele von MeterMonitor	19
4.2	Konzept	19
4.3	MeterMonitor-ESP	21
4.3.1	Programmablauf	21
4.3.2	Implementierung	23
4.3.3	Konfiguration mittels Kconfig-Datei	24
4.3.4	Weitere Optimierungen	25
4.3.5	Probleme	26
4.4	MeterMonitor-Classifer	28
5	Zusammenfassung	29
5.1	Fazit	29
5.2	Ausblick	29
	Abbildungsverzeichnis	31
	Literaturverzeichnis	32

1 Einleitung

1.1 Ziele des Projektes

Im Zuge der fortschreitenden Digitalisierung von Hochschulen und der Entwicklung von Smart-Campus-Lösungen stehen innovative Ansätze zur Optimierung von Ressourcen und Prozessen im Fokus. Ein zentraler Bestandteil solcher Konzepte ist das Auslesen und Speichern von Verbrauchsdaten, insbesondere im Bereich der Wasserversorgung. Ziel dieses Projekts ist es, die Möglichkeiten der Digitalisierung der Wasserzähler auf dem Campus zu untersuchen, um den Zählerstand der Hochschule besser nachvollziehen zu können und so einen ersten Baustein für nachhaltigere Optimierungen zu liefern.

Dabei wurde bereits zu Beginn das Open-Source-Projekt „AI-on-the-edge“ [1] als potenzielle Lösung identifiziert. Dieses Projekt nutzt einen ESP32-Cam, der in regelmäßigen Abständen Fotos der Wasserzähler aufnimmt. Die aufgenommenen Bilder werden direkt auf dem Gerät mittels eines neuronalen Netzes analysiert, um den Wasserverbrauch präzise und automatisch zu digitalisieren. Solch ein System könnte eine zentrale Rolle bei der nachhaltigen Transformation der Hochschule spielen, indem es manuelle Ablesungen ersetzt und eine kontinuierliche Überwachung des Wasserverbrauchs ermöglicht.

Das Ziel dieses Projekts ist es daher, zu evaluieren, inwiefern „AI-on-the-edge“ die Anforderungen der Anwendung am Campus erfüllt und ob es sich in die bestehende Infrastruktur integrieren lässt. Sollte sich herausstellen, dass diese Lösung nicht vollständig unseren Anforderungen entspricht, werden alternative Ansätze in Betracht gezogen, um eine effiziente und skalierbare Lösung für die Digitalisierung der Wasserzähler zu entwickeln.

Ein weiterer Fokus des Projekts liegt auf der Energieeffizienz der eingesetzten Technologien. Da in manchen Bereichen des Campus, an denen sich Wasserzähler befinden, eine dauerhafte Stromversorgung nicht sichergestellt werden kann, ist es notwendig, eine Lösung zu finden, die auch im batteriebetriebenen Modus zuverlässig funktioniert. Dies erfordert eine besonders sparsame Anwendung, um sicherzustellen, dass die Geräte über einen längeren Zeitraum hinweg autark arbeiten können, ohne dass die Batterien häufig gewechselt oder aufgeladen werden müssen.

1.2 Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich in mehrere Phasen, die schrittweise zur Realisierung der Digitalisierung der Wasserzähler auf dem Campus führen. Ein wesentlicher Aspekt des Projekts bestand darin, sowohl die Hardware als auch die Software den spezifischen Anforderungen anzupassen, um eine verlässliche Datenerfassung und Analyse zu gewährleisten.

Zunächst musste ein Gehäuse entwickelt werden, um die Hardware sicher am Wasserzähler zu befestigen. Diese mechanische Komponente ist entscheidend, da die Kamera fest positioniert sein muss, um regelmäßig präzise Bilder des Zählerstands aufnehmen zu können. Das Gehäuse erfüllt dabei nicht nur den Zweck der Befestigung, sondern schützt die Hardware auch vor Umwelteinflüssen und sorgt dafür, dass die Bilder konstant unter den gleichen Lichtbedingungen gemacht werden können.

Nachdem das Gehäuse entworfen und montiert wurde, konnte mit der Evaluierung des „AI-on-the-edge“-Projekts begonnen werden. In diesem Schritt wird untersucht, ob die vorhandene Softwarelösung für unseren spezifischen Anwendungsfall geeignet ist. Falls sich herausstellt, dass „AI-on-the-edge“ den Anforderungen nicht vollständig gerecht wird, besteht die Notwendigkeit, eine eigene Lösung zu entwickeln, um eine präzise und effiziente Digitalisierung der Wasserzähler zu gewährleisten.

Aufgrund der Abhängigkeit von einer stabilen mechanischen Befestigung wurde das Gehäuse als erster Schritt realisiert. Erst danach war es möglich, die Funktionalität und Effizienz von „AI-on-the-edge“ in der Praxis zu evaluieren. Dieser iterative Prozess bildet das Herzstück des Projekts und stellt sicher, dass sowohl die Hardware als auch die Software optimal auf den Anwendungsfall abgestimmt sind.

2 Wasserzähler-Halterung

Um das „AI-on-the-edge“-Projekt, wie im Abschnitt 1.2 beschrieben, in der Praxis sinnvoll testen zu können, war es unerlässlich, den ESP32-Cam sicher und präzise am Wasserzähler zu befestigen. Eine stabile und passgenaue Montage ist entscheidend, um wiederholbar klare Bilder des Zählerstands zu erhalten und die Daten zuverlässig auswerten zu können.

Der Entwickler von „AI-on-the-edge“ stellte hierfür ein 3D-druckbares Gehäuse zur Verfügung, das problemlos zu Hause hergestellt werden kann [2]. Dieses Gehäuse, zu sehen in Abbildung 2.1, war jedoch für die Anforderungen dieses Projekts nicht geeignet, da es auf einen festen Durchmesser des Wasserzählers ausgelegt ist und nicht flexibel angepasst werden kann. Am Campus kommen jedoch verschiedene Wasserzählermodelle zum Einsatz, was bedeutet, dass für jeden Zählertyp eine individuelle Halterung entwickelt und gedruckt werden müsste.

Ein solcher Ansatz würde die Installation erheblich verlangsamen und erschweren, da jedes Mal ein maßgeschneidertes Gehäuse erforderlich wäre. Um eine schnelle, flexible und effiziente Einrichtung der Geräte an unterschiedlichen Wasserzählern zu ermöglichen, war es daher notwendig, ein eigenes, universell einsetzbares Gehäuse zu entwickeln, das sich den Gegebenheiten unterschiedlicher Wasserzähler anpassen lässt.

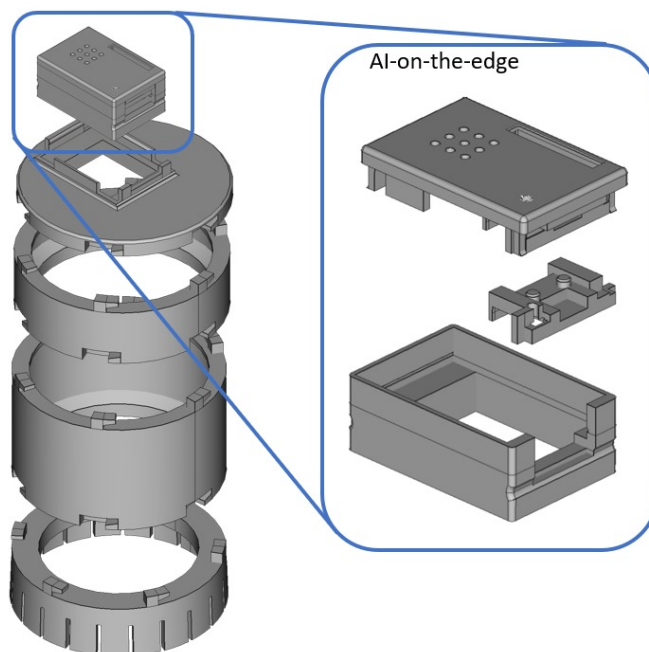


Abbildung 2.1: Aufbau der offiziellen Halterung von „AI-on-the-edge“ [2]

2.1 Konzept

Trotz der zuvor beschriebenen Nachteile des offiziellen Gehäuses von „AI-on-the-edge“ wird dieses von vielen Nutzern erfolgreich in der Praxis eingesetzt. Seine grundlegende Struktur und Funktionalität haben sich bewährt, weshalb für die Entwicklung einer neuen Halterung ein sehr ähnliches Konzept gewählt wurde. Anstatt das Gehäuse von Grund auf neu zu gestalten, wurde beschlossen, die bewährten Elemente des Designs zu übernehmen und lediglich die Halterung an den Wasserzähler zu überarbeiten.

Um die Kompatibilität mit dem bestehenden Gehäuse sicherzustellen, wurde das Gehäuse für den ESP32-Cam unverändert übernommen. Dieser Ansatz stellt sicher, dass die Kamera weiterhin zuverlässig geschützt ist und stabil arbeitet, während die neue Halterung eine flexible Anpassung an verschiedene Wasserzähler ermöglicht. Durch diese Entscheidung bleibt das System mit der ursprünglichen „AI-on-the-edge“-Halterung kompatibel, sodass eventuelle zukünftige Weiterentwicklungen oder Verbesserungen leicht integriert werden können.

Die Überarbeitung der Halterung konzentriert sich daher ausschließlich auf die Befestigung am Wasserzähler, um eine flexiblere und universellere Lösung zu schaffen, die für unterschiedliche Zählertypen geeignet ist, ohne dabei auf die bewährten Vorteile des ursprünglichen Designs zu verzichten.

In der Konzeptionsphase der neuen Halterung lag der Schwerpunkt darauf, eine Lösung zu entwickeln, die flexibel genug ist, um sich an die unterschiedlichen Wasserzähler auf dem Campus anzupassen. Um diese Anforderungen zu erfüllen, wurde entschieden, auf ein modulares Design zu setzen. Dieses Konzept bietet die nötige Flexibilität, während es eine einfache und effiziente Installation ermöglicht.

Ähnlich wie bei der ursprünglichen „AI-on-the-edge“-Halterung wurde die neue Halterung in drei Hauptkomponenten unterteilt:

1. Kamerahalterung:

Diese Komponente dient als Ankerpunkt des ursprünglichen „AI-on-the-edge“-Gehäuses (In Abbildung 2.1 als „AI-on-the-edge“ markiert). Sie sorgt dafür, dass das Gehäuse der ESP32-Cam sicher fixiert und optimal ausgerichtet ist. Da diese Komponente einen vordefinierten, unveränderlichen Durchmesser besitzt, bleibt sie universell einsetzbar und kann unabhängig vom Wasserzähler einfach ausgetauscht werden. Dadurch wird

sichergestellt, dass die Halterung flexibel bleibt und bei Änderungen am Wasserzähler ohne Weiteres weiterverwendet werden kann.

2. Verlängerungsmodul:

Um eine flexible Anpassung an verschiedene Wasserzähler zu ermöglichen, dient das Verlängerungsmodul als Bindeglied zwischen dem Wasserzähler und der Kamerahalterung. Sie sorgt dafür, dass der richtige Abstand zwischen Kamera und Zähler gewahrt bleibt. Das Modul ist an einem Ende an den Durchmesser des Wasserzählers angepasst und hat am anderen Ende einen fixen Durchmesser, der zur Kamerahalterung passt, sodass ein kegelähnlicher Körper entsteht. Zudem muss das Verlängerungsmodul geschlossen sein, um das Innere abzudunkeln, damit die Kameraaufnahmen konstant unter kontrollierten Lichtbedingungen mit der eingebauten Beleuchtung erstellt werden können.

3. Wasserzähler-Adapting:

Der Wasserzähler-Adapting dient als Verbindungsstück, das speziell auf den Durchmesser des jeweiligen Wasserzählers angepasst wird. Ein System aus flexiblen Lamellen ermöglicht dabei einen stabilen Halt am Zähler. Die Lamellen legen sich eng um den Wasserzähler und gewährleisten so eine sichere Befestigung, ohne dass zusätzliche Fixierungen nötig sind. Diese Konstruktion sorgt dafür, dass der Adapting zuverlässig am Wasserzähler sitzt und die nötige Flexibilität besitzt, um unterschiedliche Zählerdurchmesser aufzunehmen.

Das modulare Design erlaubt es, die Halterung flexibel anzupassen, ohne die Notwendigkeit, für jeden Wasserzähler ein komplett neues Gehäuse zu entwerfen. Der Wasserzähler-Adapter und das Verlängerungsmodul sind die einzigen Elemente, welche spezifisch für den jeweiligen Zähler gefertigt werden müssen, während die ESP32-Cam-Halterung und das ESP32-Cam-Gehäuse universell einsetzbar bleiben. Diese Herangehensweise sorgt dafür, dass eine effiziente und schnelle Installation an verschiedenen Zählertypen am Campus möglich ist.

Zum Verbinden der einzelnen Komponenten soll ein Schnappverschluss entwickelt werden. Dieser Ansatz ermöglicht es, ein viel kleineres Profil zu realisieren und reduziert den Materialverbrauch, da weniger Plastik benötigt wird. Da das Gehäuse in der Praxis nicht regelmäßig abgenommen werden muss, ist es nicht erforderlich, dass der Verschluss so robust ist wie im ursprünglichen Gehäuse. Dadurch wird die Konstruktion leichter und effizienter, ohne Kompromisse bei der Funktionalität einzugehen.

2.2 Modellierung

Für die Modellierung des Gehäuses wurde das CAD-Programm Fusion 360 gewählt, ein leistungsstarkes Werkzeug, das sowohl im professionellen als auch im privaten Bereich weit verbreitet ist. Studenten der Hochschule RheinMain können es kostenfrei nutzen [3].

Fusion 360 bietet eine breite Palette an Funktionen an, welche den Designprozess optimieren, darunter die einfache Erstellung von anpassbaren 3D-Modellen, die Möglichkeit zur Durchführung von Simulationen sowie die Unterstützung vieler verschiedener Exportformate. Darüber hinaus erleichtern die parametrischen Designwerkzeuge schnelle Anpassungen an den Abmessungen der Bauteile, was besonders wichtig ist, um eine passgenaue Halterung für unterschiedliche Wasserzähler zu gewährleisten. Besonders hervorzuheben ist die zeitleistenbasierte Protokollierung, die eine nachvollziehbare Dokumentation der Arbeitsschritte ermöglicht.

Begonnen wurde die Modellierung der Wasserzähler-Halterung mit einer Skizze, die die grundlegenden Maße und Dimensionen der Halterung festlegte. Diese Skizze diente als Basis für die spätere Extrusion, um das dreidimensionale Modell der Halterung zu erzeugen. Um den Durchmesser des Wasserzählers – wie zuvor besprochen – flexibel anpassen zu können, wurde die in Fusion 360 vorhandene “Parameter“-Funktion genutzt. Diese Funktion erlaubt es, anstelle von festen Werten, variable Parameter für Skizzen und Längen zu definieren, die auch nachträglich geändert werden können. Dadurch kann das Gehäuse in der finalen Datei einfach an den jeweiligen Wasserzähler-Typ angepasst werden.

Um der Halterung noch mehr Flexibilität und Anpassungsmöglichkeiten zu bieten, wurden insgesamt vier solche Parameter hinzugefügt:

- **Wasserzähler-Durchmesser:** Definiert den inneren Radius des Adapterrings, um das Design für unterschiedliche Wasserzähler anzupassen.
- **Verlängerungsmodul-Länge:** Bestimmt die Länge des Verlängerungsmoduls, sodass ein optimaler Abstand der Kamera eingehalten werden kann.
- **Anzahl der Lamellen:** Bestimmt die Anzahl der Lamellen des Adapterrings, um auch bei großen Zählern die Lamellen beweglich und elastisch zu halten.

- **Länge der Lamellen:** Definiert die Länge der Lamellen des Adapterrings, um bei unterschiedlichen Zählern trotzdem einen guten Halt zu liefern.

Mithilfe dieser Parameter kann die Halterung vor dem Drücken an unterschiedliche Wasserzähler angepasst werden.

Für die Verbindung der einzelnen Komponenten wurde sich in der Konzeptionsphase für einen Schnappverschluss entschieden. Wie der Name bereits andeutet, sollen die Komponenten durch leichtes Drücken ineinander einrasten. Der entwickelte Verschluss erstreckt sich über die gesamte Lippe des jeweiligen zylindrischen Teils. Dadurch ist es möglich, die verbundenen Teile nach dem Einrasten weiterhin zu rotieren. Dies erlaubt es, die Kamera präzise auf den Wasserzähler auszurichten, ohne dass der Schnappverschluss an Stabilität verliert.

Zusätzlich zu seiner Flexibilität reduziert der Schnappverschluss den Materialverbrauch und sorgt für ein kompaktes Design, das dennoch eine stabile Verbindung der Komponenten gewährleistet. Damit der Schnappverschluss sicher einrastet, wurde eine Kerbe in die Unterseite der entsprechenden Komponenten integriert. Diese Kerbe ist nur etwa 0,215mm tief. In der Praxis hat sich jedoch gezeigt, dass diese geringe Tiefe, obwohl sie nicht nach viel klingt, einen überraschend starken Halt bietet und die Bauteile fest und stabil miteinander verbunden bleiben.

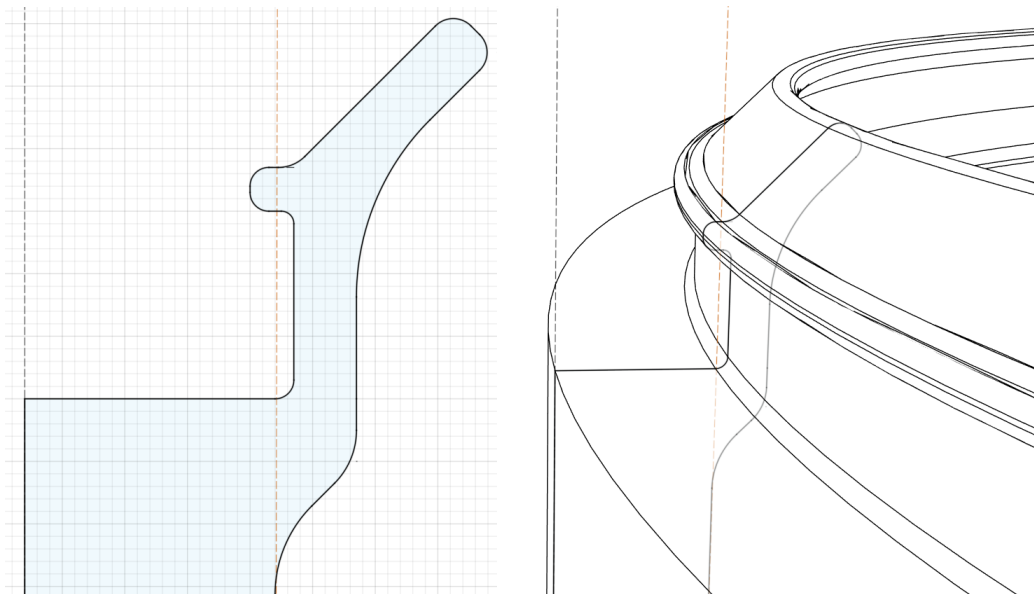


Abbildung 2.2: Entwickelter Schnappverschluss zum Verbinden der drei Hauptkomponenten

Für das oberste Modul, welches als Befestigungsmöglichkeit für den ESP32-Cam dient, wurde dieselbe Halterung wie im ursprünglichen „AI-on-the-edge“-Design übernommen. Des Weiteren wurde bei diesem Modul darauf geachtet, dass im Inneren genügend Platz für das eventuelle Einbauen eines LED-Streifens ist, falls dieser für zusätzliche Beleuchtung benötigt wird. Hierfür wurde ein 11 mm breiter Streifen ringsum unterhalb der Öffnung freigehalten, welcher genügend Platz für gängige LED-Streifen bietet.

Abschließend wurde für ein ansprechenderes Design auf den freien Flächen der Halterung ein Muster aus Sechsecken integriert. Dieses Muster verbessert nicht nur das optische Erscheinungsbild, sondern reduziert auch den Materialverbrauch beim 3D-Druck. Durch die Verwendung weniger Kunststoff bleibt das Gehäuse leicht, ohne dabei an Stabilität einzubüßen.

In der folgenden Abbildung 2.3 ist das vollständig entwickelte Gehäuse, bestehend aus den drei Komponenten, zu sehen.

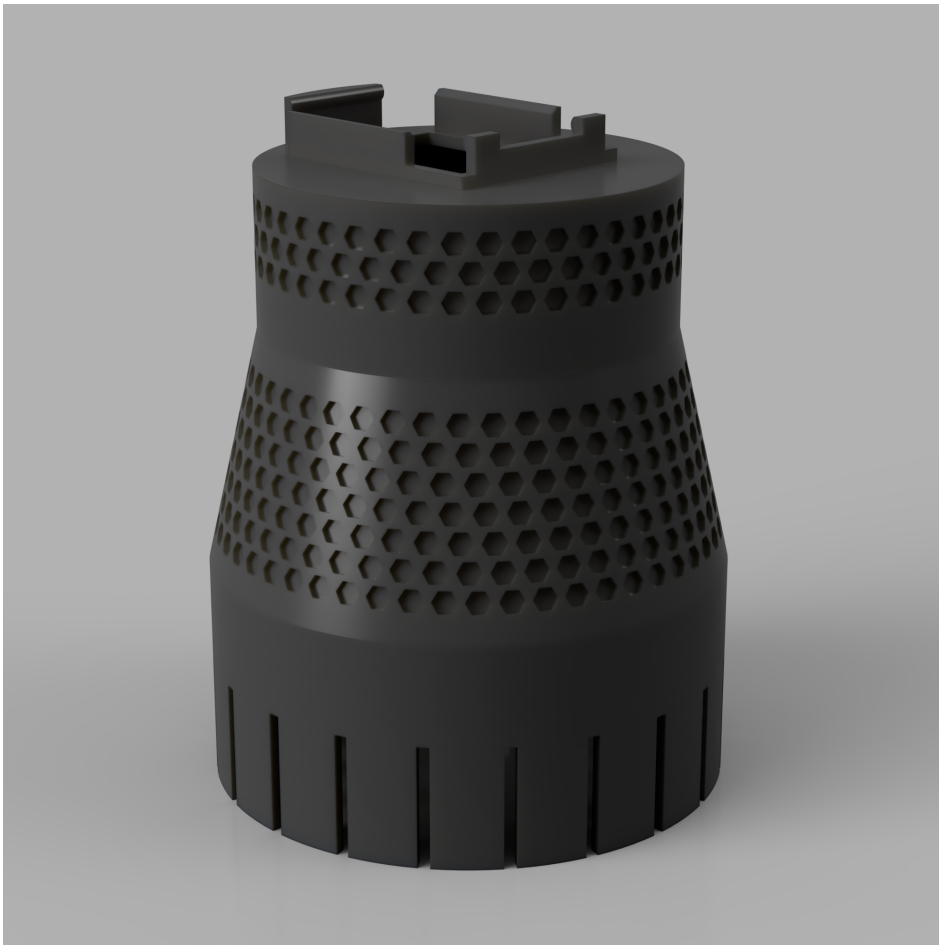


Abbildung 2.3: Beispiel des entwickelten Gehäuses für größere Wasserzähler (Bild wurde mithilfe von Ray-Tracing in Fusion 360 gerendert)

2.3 3D-Druck

Um das entwickelte Modell nun drucken zu können, müssen die Modelldateien, beispielsweise im STL- oder OBJ-Format, in geometrischen Code (G-Code) umgewandelt werden, der vom 3D-Drucker verstanden wird. Für den Druck wurde der Bambu Lab A1 FMD-Drucker verwendet, weshalb der dafür optimierte Slicer “Bambu Studio“ zum Einsatz kam. Dieses Programm bietet zahlreiche Optimierungs- und Anpassungsmöglichkeiten für den Druckprozess, wobei für den Druck dieses Modells größtenteils auf die Standardeinstellungen zurückgegriffen wurde, um eine hohe Konsistenz und Qualität mit anderen Druckern zu gewährleisten. Lediglich die Schichthöhe (Layer Height) und die Nutzung von Stützen für das Drucken der Kamerahalterung wurden angepasst, um den spezifischen Anforderungen der Halterung gerecht zu werden. Diese Änderungen trugen dazu bei, die Stabilität und Präzision des Endprodukts zu maximieren.

Durch die Form der Bauteile ist die Orientierung der einzelnen Komponenten auf dem Druckbett sofort ersichtlich, was den Druckprozess erleichtert. Das Drucken aller Komponenten, die für eine Einheit des Gehäuses aus Abbildung 2.3 notwendig sind, dauert mit dem Bambu Lab A1 in der normalen Geschwindigkeit rund 3 Stunden und 40 Minuten. Dabei werden ca. 75 g bzw. 25 m PLA-Filament mit einem Durchmesser von 1,75 mm verwendet. Bei einem gängigen Preis von 20 Euro pro Kilogramm PLA-Filament ergibt sich somit ein Materialkostenpreis von etwa 1,50 Euro für die gesamte Wasserzähler-Halterung.

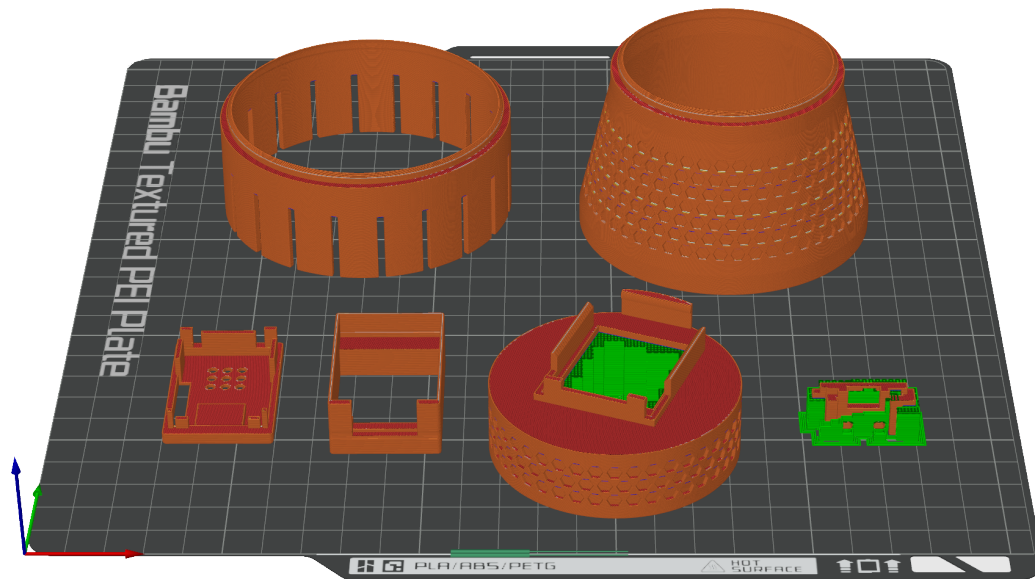


Abbildung 2.4: Geslicte Buildplate mit Stützen für die Kamerahalterung und den USB-Port

3 AI-on-the-edge

Wie bereits in Abschnitt 1.1 erwähnt, war von Anfang an geplant, das Projekt „AI-on-the-edge“ in der realen Welt zu testen und seine Eignung für unseren spezifischen Anwendungsfall zu untersuchen. Nachdem nun das Gehäuse entwickelt und so angepasst wurde, dass es auf den zu testenden Wasserzähler passt, konnte der Praxistest des Systems beginnen.

Zu Beginn wird jedoch im nächsten Abschnitt eine genauere Betrachtung des Projekts „AI-on-the-edge“ und seiner Funktionsweise vorgenommen.

3.1 Über das Projekt

Das Projekt, das offiziell unter dem Namen „AI-on-the-edge-device“ bekannt ist, wird häufig einfach als „AI-on-the-edge“ bezeichnet – auch in dieser Arbeit. Dabei handelt es sich um ein Open-Source-Projekt, welches es ermöglicht, analoge Wasser-, Gas- und Stromzähler zu digitalisieren. Im Fokus steht eine einfache und kostengünstige Umsetzung, wobei als Hardware der Ai-Thinker ESP32-Cam [4] zum Einsatz kommt, ein Mikrocontroller mit integrierter Kamera. Mit einem Kostenpunkt von weniger als 10 Euro bietet dieser eine besonders günstige Möglichkeit, Zählerwerte automatisiert zu erfassen und weiterzuverarbeiten.

Die Bilderkennung wird lokal auf dem ESP32-Cam mithilfe von TensorFlow Lite durchgeführt, ganz nach dem Prinzip des „Edge Computings“. Das bedeutet, dass die Bildverarbeitung direkt auf dem Gerät stattfindet, ohne dass ein externer Server dafür notwendig ist. Neben dem Digitalisieren von Zahlenanzeigen können auch Drehscheiben auf den Zählern erkannt und verarbeitet werden.

Dank der periodischen Bestimmung des Zählerstands lassen sich detaillierte Einblicke in den Verbrauch, beispielsweise von Wasser, über den Tag hinweg gewinnen. Diese Daten können anschließend über verschiedene Protokolle exportiert werden, darunter eine REST-API, das MQTT-Protokoll oder direkt in eine InfluxDB-Datenbank zur weiteren Analyse und Visualisierung.

Die Kombination aus kostengünstiger Hardware und leistungsfähiger Software macht „AI-on-the-edge“ zu einer attraktiven Lösung für die Digitalisierung von Zählerständen, insbesondere im privaten Umfeld.

Die folgende Abbildung 3.1 zeigt ersichtlich den Workflow, welcher jeweils im konfigurierten Zeitintervall durchgeführt wird.

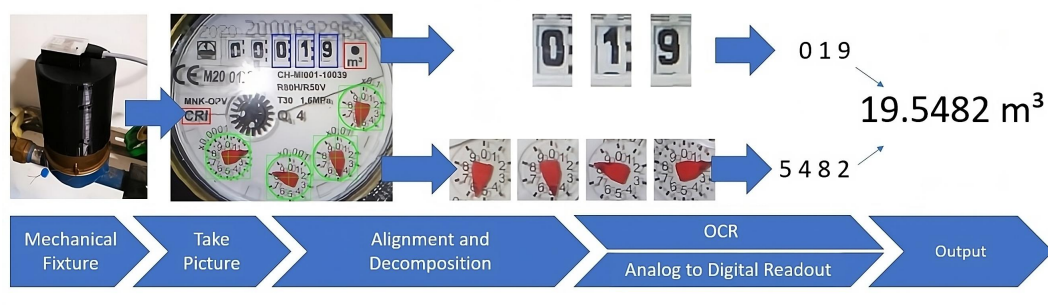


Abbildung 3.1: Schematischer Arbeitsablauf von „AI-on-the-edge“

Zu beachten ist, dass für das Evaluieren im Rahmen dieser Arbeit nur ein Wasserzähler zur Verfügung stand, der ausschließlich eine Zahlenanzeige und keine Drehscheiben enthielt. Aus diesem Grund konnte lediglich das Tensor-Flow Lite-Modell zur Erkennung der Zahlen getestet werden. Die Erkennung von Drehscheiben, die ebenfalls von „AI-on-the-edge“ unterstützt wird, blieb ungetestet.

3.2 Inbetriebnahme

Für die Inbetriebnahme von „AI-on-the-edge“ sind die folgenden drei Schritte erforderlich, um die Funktionalität des Geräts zu gewährleisten. Diese Schritte müssen bei mehreren Wasserzählern für jede Einheit jeweils manuell durchgeführt werden.

3.2.1 Flashen des ESP32-Cam

Um die Inbetriebnahme von „AI-on-the-edge“ zu beginnen, muss zunächst der ESP32-Cam geflasht werden. Bei ESPs mit einem USB-Anschluss erfolgt dies in der Regel problemlos über die integrierte USB-Schnittstelle. Der ESP32-Cam hingegen besitzt keinen USB-Anschluss und somit auch keinen integrierten USB-Controller-Chip, der für die direkte USB-Kommunikation zuständig wäre. Stattdessen muss die serielle Schnittstelle (UART) für die Kommunikation mit dem Computer genutzt werden.

Hierzu wird ein USB-Serial-Adapter benötigt, welcher die Verbindung zwischen dem ESP32-Cam und dem Computer ermöglicht. Der Mikrocontroller muss dabei, wie in Abbildung 3.2 dargestellt, angeschlossen werden. Dabei ist es wichtig, dass der GPIO0-Pin beim Booten mit GND verbunden wird. Dies ist erforderlich, damit der ESP in den Bootloader-Modus wechselt, wodurch der

Programmcode von „AI-on-the-edge“ auf das Gerät übertragen werden kann.

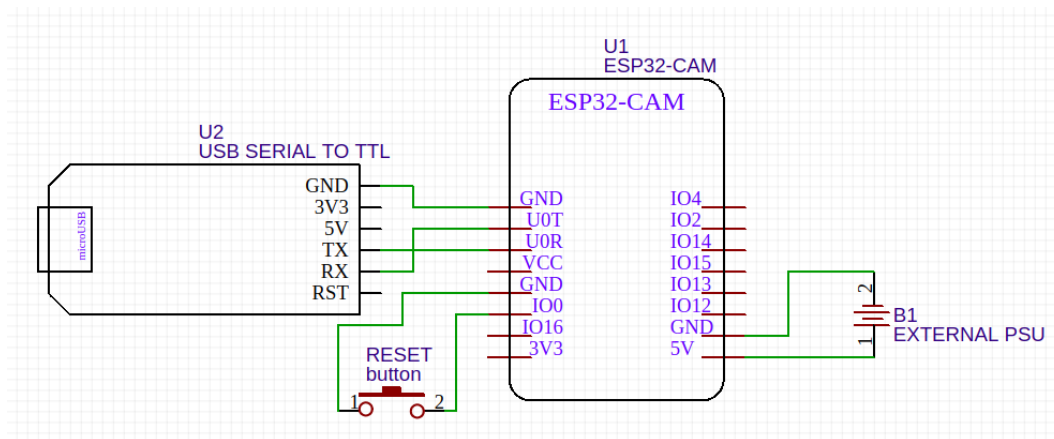


Abbildung 3.2: Flashen des ESP32-Cam mithilfe eines USB-Serial-Adapters [5]

Nachdem der ESP32-Cam korrekt verkabelt wurde, kann der Flashvorgang mittels der dafür zur Verfügung stehenden [Webseite](#) aus dem Browser gestartet werden. Dabei ist zu bedenken, dass dieser Vorgang beim Testen ausschließlich mit Google Chrome erfolgreich war - andere Browser wie Firefox oder Chromium-basierte Browser wie Brave konnten keine Verbindung mit dem USB-Serial-Adapter aufbauen.

Nach dem initialen Flashvorgang wird der USB-Serial-Adapter nicht mehr benötigt, da „AI-on-the-edge“ die Möglichkeit bietet, Updates bequem per Over-the-Air (OTA) zu installieren.

Neben der Installation auf dem ESP32-Cam ist es außerdem notwendig, die erforderlichen Daten auf die SD-Karte zu übertragen. Diese Daten können bequem von der [Dokumentations-Seite](#) heruntergeladen und auf eine entsprechend formatierte SD-Karte kopiert werden. Während dieses Vorgangs sollte auch die Datei „wlan.ini“ angepasst werden, um die SSID und das Passwort des gewünschten Netzwerks einzutragen.

Mit dem Einsetzen der SD-Karte in den ESP32-Cam ist die grundlegende Vorbereitung abgeschlossen. Bevor jedoch der ESP in die Halterung auf dem Wasserzähler montiert werden kann, muss noch die Kamera passend modifiziert werden.

3.2.2 Anpassung der Brennweite für scharfe Nahaufnahmen

Der ESP32-Cam wird mit der Kamera OV2640 ausgeliefert, die mit einer Linse ausgestattet ist, die primär für weiter entfernte Aufnahmen konzipiert ist. Aus diesem Grund hat die Linse eine minimale Brennweite von über 40 cm. Für „AI-on-the-edge“ bedeutet das, dass der Kameraabstand zum Wasserzähler sehr groß sein müsste, was dazu führen würde, dass der Zähler nur einen kleinen Teil des Bildes einnimmt. Um dies zu optimieren, muss der ESP32-Cam näher am Wasserzähler montiert werden. Diese Nähe führt jedoch zu unscharfen Bildern, wenn die Brennweite nicht angepasst wird.

Die Brennweite der Kamera kann glücklicherweise jedoch durch Drehen der Linse verändert werden. Zunächst muss der Kleber, der die Linse an der Kamera befestigt, mit einem scharfen Messer vorsichtig abgeschabt werden. Nach dieser Vorbereitung kann die Linse im Uhrzeigersinn gedreht werden, um den Fokus für Nahaufnahmen anzupassen.

Diese Anpassung gestaltet sich oft als schwierig, da die Kamera sehr klein ist und der Kleber hartnäckig haftet. Allerdings wurden zwei 3D-druckbare Modelle im Internet gefunden, die diesen Prozess erheblich erleichtern. Eines der Modelle dient zum Fixieren der Kamera, während die Linse modifiziert wird, und das andere erleichtert das präzise Drehen der Linse. Beide Modelle sind in der folgenden Abbildung 3.3 zu sehen.



Abbildung 3.3: Links: ESP32-CAM / OV2640 lens wrench (von: mkarliner) [6]
Rechts: ESP32-CAM Focus tool (von: AskePetter) [7]

Nach Abschluss dieser Modifikation ist der ESP32-Cam nun bereit, in der Halterung am Wasserzähler montiert und für den Einsatz vorbereitet zu werden.

3.2.3 Initiale Konfiguration

Nachdem der ESP32-Cam erfolgreich mit der Firmware geflasht, die WLAN-Zugangsdaten korrekt eingetragen und die SD-Karte eingerichtet wurde, hostet das Gerät einen lokalen Webserver, über den die initiale Konfiguration durchgeführt wird.

In dieser wird zuerst ein Referenzbild erstellt, anhand dessen grundlegende Kameraeinstellungen wie Helligkeit und Sättigung optimiert werden können. Im nächsten Schritt werden auf diesem Bild zwei Referenzmarker gesetzt, die helfen, das Bild korrekt auszurichten. Sobald das Bild referenziert ist, werden in den folgenden Schritten die digitalen und analogen Region-of-Interest (ROIs) explizit definiert, welche die Bereiche markieren, in denen die Ziffern oder Drehscheiben des Wasserzählers erkannt werden sollen.

Zum Schluss lassen sich verschiedene Optionen wie der Gerätename, das Messintervall sowie die Übertragung der Daten an einen MQTT-Server konfigurieren. Nach Abschluss dieser Schritte wechselt das Gerät in den normalen Betriebsmodus und beginnt, den Wasserzählerstand in den festgelegten Intervallen zu erfassen.

Es ist ebenfalls möglich, all diese Einstellungen auch nach der initialen Einrichtung über den lokalen Webserver im laufenden Betrieb anzupassen. Dadurch kann die Konfiguration flexibel geändert werden, ohne dass das Gerät neu geflasht oder zurückgesetzt werden muss.

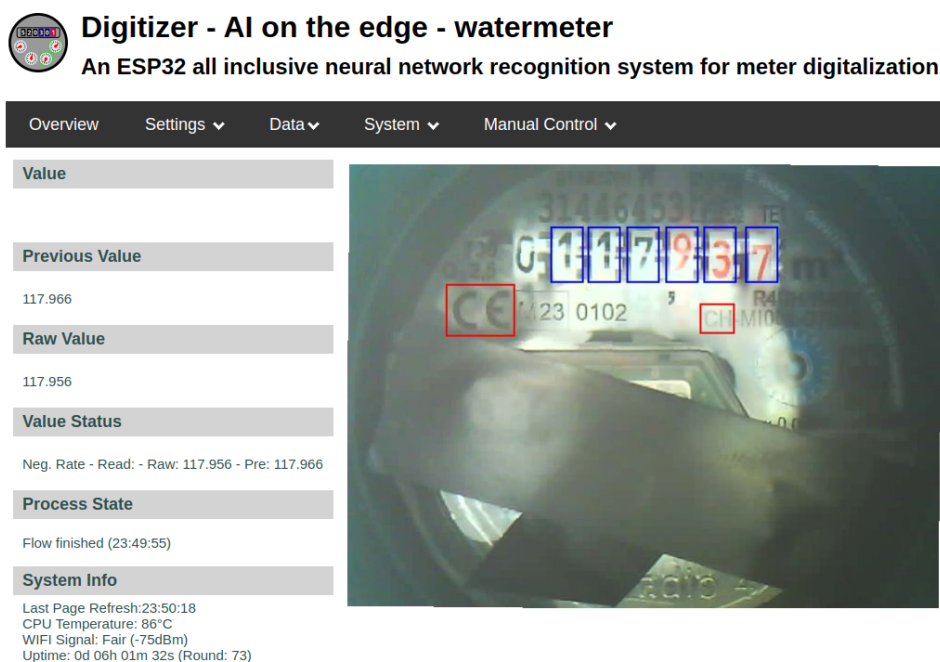


Abbildung 3.4: Weboberfläche von „AI-on-the-edge“ im Normalbetrieb

3.3 Betrieb von „AI-on-the-edge“

Im Rahmen dieser Arbeit wurde eine Instanz von „AI-on-the-edge“ über mehrere Wochen hinweg anhand eines Wohnungs-Wasserzählers in der Praxis getestet. Zu Beginn stellte die Internetverbindung des ESP32-Cam im Keller, wo der Wasserzähler positioniert war, eine Herausforderung dar. Zwar verfügt der ESP32-Cam über eine kleine eingebaute Antenne, die grundsätzlich eine WiFi-Verbindung herstellen kann, jedoch erwies sich diese aufgrund ihrer geringen Größe und der ungünstigen Positionierung als ungeeignet, um eine stabile Verbindung zum Router aufrechtzuerhalten.

Glücklicherweise besitzt der ESP32-Cam einen IPEX-Anschluss, über den eine externe Antenne angeschlossen werden kann. Um diese externe Antenne zu verwenden, musste allerdings ein winziger Widerstand auf der Platine umgelötet werden, damit die Antennenverbindung entsprechend umzuschalten. Nachdem die externe Antenne installiert war, konnte eine deutlich stabilere und zuverlässigere Internetverbindung im Kellerbereich hergestellt werden, was den reibungslosen Betrieb von „AI-on-the-edge“ sicherstellte.



Abbildung 3.5: Testaufbau von „AI-on-the-edge“ mit externer Antenne

Um möglichst viele Iterationen durchzuführen und umfassende Erfahrungen mit „AI-on-the-edge“ zu sammeln, wurde in der Testphase ein relativ kurzes Messintervall von 5 Minuten gewählt. Die erfassten Werte wurden dank des eingebauten MQTT-Protokolls an einen lokal selbst gehosteten Broker gesen-

det und in eine Home Assistant-Instanz eingespeist. Der Datenexport und die Integration in Home Assistant verliefen dabei ohne jegliche Probleme, was die Zuverlässigkeit der Software in dieser Hinsicht bestätigte. Auch das sonstige Layout der Webseite und die Bedienung sind sehr gut durchdacht.

Zunächst wurde das Gerät während der Testphase über eine normale Steckdose mit Strom versorgt. Erst später wurden Tests mit verschiedenen Powerbanks durchgeführt, um den Stromverbrauch zu überprüfen. Weitere Details dazu werden im Laufe des folgenden Kapitels 3.4 näher erläutert.

3.4 Herausforderungen und Probleme

Im Laufe der Testphase wurden mehrere Probleme und Herausforderungen identifiziert, die bei einem großflächigen Deployment von „AI-on-the-edge“ an der Hochschule problematisch sein könnten. Im Folgenden werden die drei größten Probleme und Herausforderungen nach ihrer Relevanz aufsteigend geordnet dargelegt:

Fehlende Authentifizierung auf dem Webserver

Ein großflächiges Deployment von „AI-on-the-edge“ an der Hochschule würde erfordern, dass alle ESP32-Cams in das Campus-Netzwerk integriert werden. Ein zentrales Problem dabei ist, dass die von den ESPs gehosteten Webserver ungeschützt sind und keinerlei Zugangskontrolle bieten. Dies bedeutet, dass jeder, der sich im selben Netzwerk befindet, auf die Weboberfläche der Geräte zugreifen und sämtliche Einstellungen und Konfigurationen verändern könnte. Da die Weboberfläche es ermöglicht, wichtige Parameter zu modifizieren, würde ein offener Zugriff auf diese Schnittstelle das Risiko von unbefugten Änderungen erheblich erhöhen.

Eine mögliche Lösung wäre, die ESPs in ein privates Netzwerk zu integrieren, das nur von vertrauenswürdigen Teilnehmern genutzt wird. Dennoch ist dieser Mangel an Zugriffskontrolle eine wesentliche Herausforderung, die bei einem breiteren Einsatz berücksichtigt und gelöst werden sollte.

Mangelhafte Beleuchtung und Spiegelungen

Ein weiteres grundlegendes Problem bei der Nutzung von „AI-on-the-edge“ ist die Beleuchtung für die Aufnahme der Bilder. Der ESP32-Cam ist zwar mit einer eingebauten, starken LED ausgestattet, jedoch befindet sich diese LED aufgrund der Bauweise sehr nah an der Kamera. Da sowohl die Kamera als

auch die LED direkt über der reflektierenden Oberfläche des Wasserzählers platziert sind, führt dies dazu, dass sich das Licht der LED auf der glänzenden Fläche des Zählers spiegelt. Diese Spiegelungen machen Teile des Zählers auf den aufgenommenen Bildern unleserlich.

Um dieses Problem zu beheben oder zumindest zu minimieren, wurden verschiedene Lösungsansätze ausprobiert. Nachdem die Optimierung der Kameraeinstellungen keine signifikanten Verbesserungen brachte, wurde versucht, das Licht des eingebauten Flash besser zu streuen. Dazu wurde eine Schicht weißes Backpapier zwischen die Kamerahalterung und das Verlängerungsmodul geklemmt. In die Mitte des Backpapiers wurde ein Loch geschnitten, das groß genug war, um der Kamera weiterhin eine klare Sicht auf den Wasserzähler zu ermöglichen. Diese einfache Maßnahme reduzierte die Spiegelungen, allerdings nur in begrenztem Umfang.

Um das Problem weiter zu minimieren, wurde schließlich ein externer LED-Streifen installiert, der im Deckel der Kamerahalterung befestigt wurde. „AI-on-the-edge“ unterstützt hierfür den weit verbreiteten WS2812B LED-Streifen. Das Innere der Kamerahalterung wurde mit 9 LEDs des Streifens ausgestattet, um eine gleichmäßigere Verteilung des Lichts zu erreichen.

Die folgende Abbildung 3.6 zeigt die vier getesteten Varianten: Aufnahmen mit dem Flash sowie mit dem LED-Streifen, jeweils mit und ohne den Diffusor.

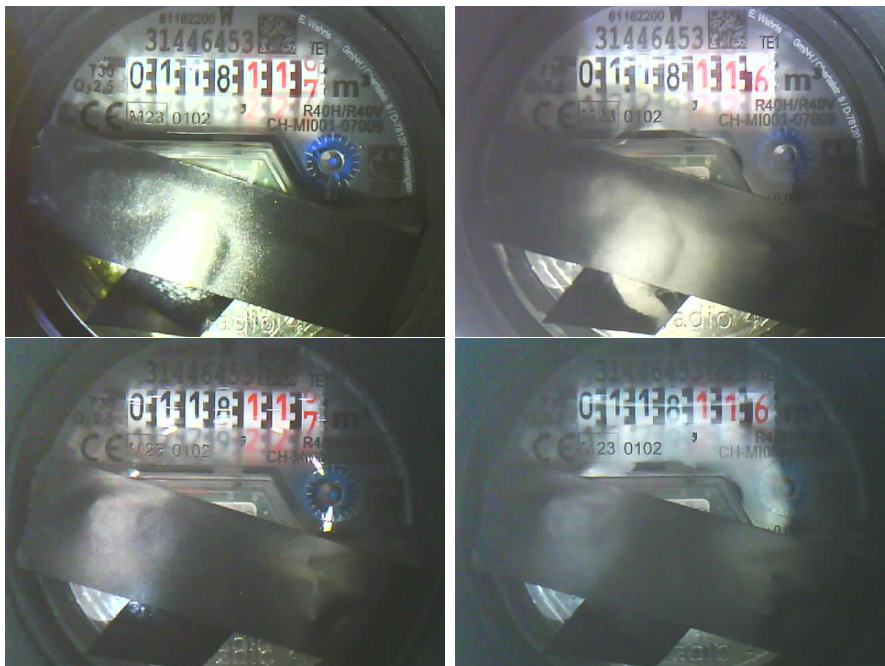


Abbildung 3.6: oben: Eingebauter Flash (links ohne, rechts mit Diffusor)
unten: Externer LED-Streifen (links ohne, rechts mit Diffusor)

Wie man in diesem Vergleich sieht, sind trotz aller Bemühungen in allen Bildern weiterhin Spiegelungen auf den Zahlen zu erkennen. Dennoch ist die Spiegelung mit dem LED-Streifen und dem Backpapier-Diffusor am geringsten ausgeprägt, weshalb „AI-on-the-edge“ in dieser Konfiguration die besten Ergebnisse lieferte. Besonders die mittlere Ziffer des Wasserzählers war jedoch während der Testphase anfällig für Fehler, da sich hier die meisten Spiegelungen sammelten. Die häufigsten Fehler bestanden darin, dass ähnliche Ziffern, wie 1 und 7 oder 6 und 8, falsch erkannt wurden. Insgesamt waren die Ergebnisse des OCRs jedoch zu häufig fehlerhaft, weshalb bei einem großflächigen Deployment auf jeden Fall ein Postprocessing und eine Validierung der Daten erforderlich wären, um die Genauigkeit der Messergebnisse sicherzustellen.

Hoher Stromverbrauch

Das größte Problem in Bezug auf ein Deployment an der Hochschule ist der hohe Stromverbrauch. Wie bereits in Abschnitt 1.1 beschrieben, ist eine der grundlegenden Anforderungen des Projekts ein geringer Energieverbrauch, da nicht davon ausgegangen werden kann, dass eine dauerhafte Stromversorgung an allen Wasserzählern verfügbar ist. In der Testphase wurde die Akkulaufzeit anhand zweier unterschiedlicher Powerbanks getestet. Ein 5.000mAh-Akku hielt dabei nur etwa 48 Stunden, was auf einen durchschnittlichen Verbrauch von rund 100mA@5V schließen lässt. Auch die zweite Powerbank lieferte einen sehr ähnlichen durchschnittlichen Verbrauch.

Dieses Ergebnis ist nicht überraschend, da einer der größten Stromfresser des ESP32-Cams das Netzwerkinterface dessen ist, das kontinuierlich aktiv bleibt, um die Netzwerkverbindung aufrechtzuerhalten. Besonders in der Testphase, in der ein sehr kurzes Messintervall von nur 5 Minuten verwendet wurde, zeigte sich der hohe Stromverbrauch. Zwar könnte dieses Intervall in der Praxis deutlich höher angesetzt werden, doch selbst wenn kein Bild analysiert wird, bleibt der Stromverbrauch durch die dauerhafte WLAN-Verbindung hoch.

Hinzu kommt der erhöhte Stromverbrauch beim Aufnehmen der Bilder sowie bei der anschließenden Verarbeitung durch das neuronale Netz. Während dieser Phase läuft der ESP32-Cam mit seiner maximalen Taktrate von 240 MHz, und dennoch dauert die Digitalisierung der sechs Ziffern etwa eine Minute, während welcher viel Energie verbraucht wird.

Aus diesen Gründen ist ein flächendeckendes Deployment von „AI-on-the-edge“ an der Hochschule in der aktuellen Form nicht praktikabel. Eine alternative Lösung mit effizienterem Stromverbrauch ist daher notwendig.

4 MeterMonitor

Wie in Abschnitt 3.4 beschrieben, bringt das Open-Source-Projekt „AI-on-the-edge“ einige Herausforderungen mit sich, die ein großflächiges Deployment an der Hochschule erschweren. Insbesondere der hohe Stromverbrauch macht es für unseren Anwendungsfall unbrauchbar, da es nicht realistisch ist, an allen Wasserzählern eine dauerhafte Stromversorgung bereitzustellen. Da es keine geeignete Open-Source-Lösung gibt, die auf kostengünstiger Hardware wie dem ESP32-Cam funktioniert und gleichzeitig unsere Anforderungen erfüllt, wurde die Entscheidung getroffen, eine eigene Lösung zu entwickeln.

Aus diesem Grund entstand „MeterMonitor“, ein neues Projekt, das speziell für unseren Anwendungsfall entwickelt wurde und die Herausforderungen, die bei „AI-on-the-edge“ aufgetreten sind, adressiert.

4.1 Ziele von MeterMonitor

Das Hauptziel von „MeterMonitor“ besteht darin, den größten Nachteil von „AI-on-the-edge“, nämlich den hohen Stromverbrauch, zu beseitigen und so einen Batteriebetrieb zu ermöglichen. Dabei soll weiterhin auf den ESP32-Cam gesetzt werden, um das bereits entwickelte Gehäuse auch für „MeterMonitor“ nutzen zu können. Um einen niedrigen Energieverbrauch zu erreichen, soll der energiesparende DeepSleep-Modus des ESP32-Cam verwendet werden.

Natürlich bleiben die in Abschnitt 1.1 definierten Ziele weiterhin von zentraler Bedeutung. Der Wasserzähler soll nach wie vor zuverlässig digitalisiert werden, um den Wasserverbrauch effizient und genau zu erfassen. Alle weiteren Anforderungen, wie eine einfache Handhabung und kostengünstige Umsetzung, gelten auch für „MeterMonitor“.

4.2 Konzept

Das Hauptziel von „MeterMonitor“ ist es, den Stromverbrauch so gering wie möglich zu halten. Aus diesem Grund wird die Architektur des Systems entsprechend angepasst, um die Effizienz zu maximieren.

Ein wesentlicher Aspekt des neuen Konzepts besteht darin, die digitale Verarbeitung der Zählerdaten nicht mehr direkt auf dem ESP32-Cam durchzuführen. Stattdessen wird diese Aufgabe auf einem leistungsstärkeren Server zen-

tralisiert. Dadurch werden die Anforderungen an den ESP erheblich reduziert, was nicht nur den Stromverbrauch minimiert, sondern auch die Notwendigkeit eines Webservers als Frontend für die Benutzer eliminiert.

Die vom ESP32-Cam aufgenommenen Bilder werden in Gänze über das MQTT-Protokoll an den zentralen Server übertragen. Dies ermöglicht dem ESP, so viel Zeit wie möglich im energieeffizienten DeepSleep-Modus zu verbringen, was entscheidend für den Batteriebetrieb ist. Um die volle Kontrolle über die Funktionsweise des ESP zu behalten und den Stromverbrauch weiter zu optimieren, wird der Code mithilfe des ESP-IDF-Frameworks entwickelt. Durch die Nutzung von ESP-IDF ist es möglich, auf detaillierte Hardware-Einstellungen zuzugreifen und so den Energieverbrauch des ESP32-Cam gezielt zu reduzieren.

Durch die Verarbeitung der Bilder auf einem leistungsstarken Computer anstelle des ESP32-Cam können zudem bessere und rechenintensive Modelle für das OCR eingesetzt werden. Eine zentrale Anlaufstelle für die Verarbeitung der Bilder ist besonders vorteilhaft, da an der Hochschule viele Wasserzähler vorhanden sind. Dies erleichtert das Instandhalten und Aktualisieren der Software erheblich, sodass das System effizienter und skalierbarer betrieben werden kann.

Für dieses Projekt wurden insgesamt drei GitLab-Repositorys erstellt, die jeweils unterschiedliche Teile von „MeterMonitor“ abdecken und die Entwicklung und Wartung des Systems erleichtern. Diese Repositorys sind wie folgt aufgeteilt:

Repository	Beschreibung
MeterMonitor-ESP	Beinhaltet den Code, der auf dem ESP32-Cam läuft. Dieser Code steuert das Aufnehmen von Bildern und das Senden dieser per MQTT. Der Code wird mithilfe von ESP-IDF entwickelt, um in der Lage zu sein, den Stromverbrauch zu optimieren.
MeterMonitor-Classifer	Enthält den Code, der zentral auf dem Server ausgeführt wird. Dieser in Python geschriebene Code empfängt die Bilder vom ESP32-Cam und wertet sie mithilfe von OCR aus.
MeterMonitor-Fixture	Enthält alle druckbaren Modelle des zuvor entwickelten Gehäuses und weitere hilfreiche 3D-Modelle für das Projekt.

Tabelle 4.1: Übersicht der Repositorys von „MeterMonitor“

4.3 MeterMonitor-ESP

Wie im vorherigen Abschnitt 4.2 beschrieben, zielt „MeterMonitor-ESP“ darauf ab, eine Anwendung zu entwickeln, die auf ESP-IDF (Espressif IoT Development Framework) basiert. Diese Applikation wurde speziell dafür entworfen, die Kamera eines ESP32-basierten Boards für das Aufnehmen und Versenden von Bildern des Wasserzählers effizient zu nutzen.

Durch die Verwendung des DeepSleep-Modus wird der ESP32-Cam in Phasen, in denen keine Aktivität erforderlich ist, in einen äußerst stromsparenden Zustand versetzt. Dies ermöglicht einen längeren Betrieb im Batteriemodus, wie in den Zielvorgaben von MeterMonitor gefordert. Sobald das Gerät aktiviert wird, nimmt es ein Bild des Wasserzählers auf und überträgt dieses mithilfe des MQTT-Protokolls an den zentralen Server zur Verarbeitung.

Im folgenden Unterkapitel 4.3.1 wird hierfür der genaue Programmablauf der Applikation detailliert beschrieben.

4.3.1 Programmablauf

Dieses Unterkapitel beschreibt die wesentlichen Schritte, die das Programm von „MeterMonitor-ESP“ durchläuft, um die Anforderungen an einen möglichst stromsparenden und effizienten Betrieb zu erfüllen. Das Programm ist so gestaltet, dass der ESP32-Cam während seiner aktiven Zeit nur die notwendigsten Aufgaben übernimmt, bevor er wieder in den energieeffizienten DeepSleep-Modus wechselt.

Um ein Bild vom Wasserzähler zu erfassen und zu übertragen, muss der ESP32-Cam mehrere zentrale Aufgaben bewältigen. Zunächst wird beim Aufwachen aus dem DeepSleep die WLAN-Verbindung aufgebaut, damit der ESP sich mit dem lokalen Netzwerk und dem MQTT-Broker verbinden kann. Mithilfe des [“ESP32 Camera Drivers“](#) [8] wird die Kamera passend initiiert, um ein Foto vom Wasserzähler zu machen. Über MQTT wird dieses Bild später an den vorausgewählten Broker geschickt, wo die eigentliche digitale Auswertung der Zählerstände erfolgt.

Ein weiteres Element des Programms ist die Synchronisation der Systemzeit mithilfe eines NTP-Zeitservers. Diese Synchronisierung ist notwendig, um die korrekte zeitliche Zuordnung der aufgenommenen Bilder sicherzustellen. Sie ermöglicht es dem System außerdem, periodische Aufnahmen exakt nach einem festen Intervall vorzunehmen.

Im Folgenden ist der Programmablauf von „MeterMonitor-ESP“ in Form eines

vereinfachten Pseudocodes dargestellt, der die grundlegenden Schritte verdeutlicht:

```
1 RTC_DATA_ATTR static struct timeval sleep_enter_time;
2 static struct timeval wake_up_time;
3 RTC_DATA_ATTR int bootCount;
4
5 static void picture_capture_task() {
6     gettimeofday(&wake_up_time, NULL);
7     ++bootCount;
8     // ----- INIT WIFI -----
9         [...]
10    // ----- Sync Time IF needed -----
11        [...]
12    // ----- INIT MQTT -----
13        [...]
14    // ----- TAKE PICTURE -----
15        [...]
16    // ----- SEND PICTURE -----
17        [...]
18    // ----- DE-INIT CAMERA & MQTT & WIFI -----
19        [...]
20    // ----- START SLEEP -----
21    gettimeofday(&sleep_enter_time, NULL);
22    start_deep_sleep();          // Setzt Timer passend
23 }
```

Code 4.1: Programmverlauf von „MeterMonitor-ESP“

Des Weiteren ist anzumerken, dass für das Aufnehmen von Fotos weiterhin die Verwendung einer Beleuchtung, wie dem eingebauten Flash des ESP32-Cam oder eines externen WS2812B-LED-Streifens, vorgesehen ist. Dabei muss jedoch darauf geachtet werden, dass diese Lichtquellen nur so kurz wie möglich eingeschaltet bleiben, um den Stromverbrauch minimal zu halten. Da LEDs, insbesondere leistungsstarke wie die WS2812B, erhebliche Mengen an Energie verbrauchen können, wäre ein unnötig langes Einschalten kontraproduktiv für den energieeffizienten Betrieb von MeterMonitor.

4.3.2 Implementierung

Die Implementierung von „MeterMonitor-ESP“ begann mit der zentralen Aufgabe, den Stromverbrauch durch den Einsatz des DeepSleep-Modus zu minimieren. In ESP-IDF gibt es mehrere Optionen, um den ESP32-Cam aus dem DeepSleep zu wecken: Timer, Touchpad, externe Wakeup-Trigger oder den ULP-Coprozessor-Wakeup. Für den Anwendungsfall des regelmäßigen Aufwachens in festen Intervallen erwies sich der Timer als die beste Wahl.

Um das Messintervall möglichst exakt einzuhalten und zeitliche Abweichungen zu vermeiden, wird der Zeitpunkt des Einschlafens und des Aufwachens aufgezeichnet. Dadurch lässt sich die Dauer der aktiven Phase berechnen, und diese wird bei der Timer-Einstellung berücksichtigt. So wird sichergestellt, dass der ESP immer genau zur geplanten Zeit aufwacht, ohne dass sich allmählich ein Fehler durch Zeitverschiebungen einschleicht.

Im Pseudocode von 4.1 wird vereinfacht gezeigt, wie diese Mechanik umgesetzt wird. Hier kommt die `RTC_DATA_ATTR`-Anweisung zum Einsatz, um die nötigen Variablen im RTC-Speicher zu behalten. Dieser Speicher bleibt auch während des DeepSleep-Modus aktiv, sodass Daten wie der Schlafzeitpunkt und die Anzahl der bisher aufgenommenen Fotos auch nach dem Aufwachen verfügbar bleiben.

Für die Steuerung der eingebauten Flash-LED des ESP32-Cam kann der entsprechende GPIO-Pin direkt angesprochen werden. Hierzu wird mithilfe von `gpio_set_level(FLASH_LED_GPIO, 1)` die LED eingeschaltet, und durch Setzen des Pins auf 0 wieder ausgeschaltet. Für die Ansteuerung eines WS2812B LED-Streifens ist jedoch eine komplexere Lösung erforderlich, da dieser per SPI betrieben wird und ein genaues Timing benötigen. Hierfür wurde Code der Open-Source-Bibliothek „esp_ws28xx“ [9] verwendet, die die Kommunikation mit diesen LEDs ermöglicht und deren Ansteuerung vereinfacht.

Nach dem Aufnehmen des Fotos müssen sowohl das Bild als auch zusätzliche Informationen, wie der Zeitstempel und der Name des Wasserzählers, über das MQTT-Protokoll übertragen werden. Um diese Daten effizient zu verpacken, wurde das JSON-Format gewählt. Mithilfe der cJSON-Bibliothek werden die Informationen strukturiert und in ein JSON-Objekt konvertiert. Da das Foto im JPEG-Format vorliegt, wird es für die Übertragung per MQTT in eine Base64-codierte Zeichenkette umgewandelt, um eine problemlose Übermittlung sicherzustellen.

Das für „MeterMonitor“ verwendete JSON-Format, das von jeder Instanz an den MQTT-Broker gesendet wird, sieht wie folgt aus:

```
1 {
2     "name": "Beispiel-Wasserzaehler",
3     "picture_number": 57,
4     "WiFi-RSSI": -71,
5     "picture": {
6         "format": "jpeg",
7         "timestamp": "2024-09-25T20:00:00",
8         "width": 640,
9         "height": 480,
10        "length": 17999,
11        "data": "/9j/4AAQSkZJRgABAQEAAAAAAAAAD/2w[...]"
12    }
13 }
```

Code 4.2: Aufbau des „MeterMonitor“ JSON-Pakets

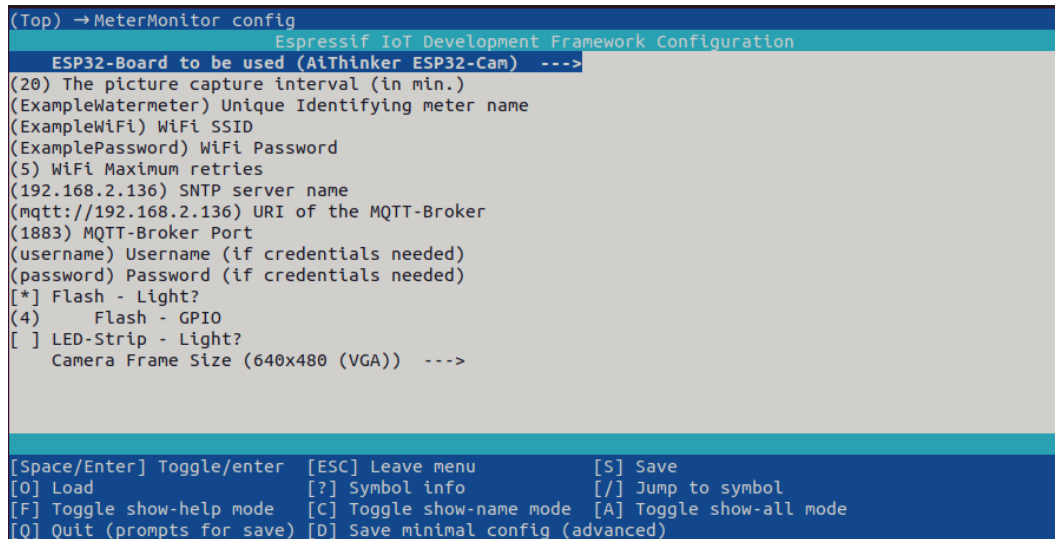
Um Fehlerquellen zu minimieren und die aktive Laufzeit des ESP weiter zu verkürzen, wurde darauf verzichtet, die Metadaten wie Timestamp oder die Anzahl der aufgenommenen Fotos in die EXIF-Felder des JPEGs zu speichern. Stattdessen wird diese Aufgabe dem „MeterMonitor-Classifer“ auf dem Server überlassen, der diese Daten nach dem Empfang der Bilder verarbeiten und entsprechend speichern kann.

4.3.3 Konfiguration mittels Kconfig-Datei

Um die Konfiguration von „MeterMonitor-ESP“ so einfach wie möglich zu gestalten, was für ein größeres Deployment an der Hochschule essenziell ist, wurden alle relevanten Systemeinstellungen in einer Kconfig-Datei zusammengefasst. Dies ermöglicht eine zentrale, übersichtliche Verwaltung der Einstellungen, die in ESP IDF bequem über eine grafische Benutzeroberfläche in der Kommandozeile konfiguriert werden kann.

Durch den Aufruf von `idf.py menuconfig` können Nutzer in der erscheinenden GUI unter dem Menüpunkt „MeterMonitor config“ alle nötigen Konfigurationen vornehmen. Dazu gehören beispielsweise das Festlegen von MQTT-Einstellungen, das Intervall der Fotoaufnahmen, sowie die WiFi-Zugangsdaten.

Diese einfache Handhabung stellt sicher, dass auch bei einem großflächigen Einsatz an der Hochschule die Konfiguration für jedes Gerät standardisiert und leicht anpassbar bleibt. Ein Beispiel für die Konfigurationsoberfläche ist in folgender Abbildung 4.1 zu sehen:



```

(Top) → MeterMonitor config
Espressif IoT Development Framework Configuration
ESP32-Board to be used (AiThinker ESP32-Cam) --->
(20) The picture capture interval (in min.)
(ExampleWatermeter) Unique Identifying meter name
(ExampleWiFi) WiFi SSID
(ExamplePassword) WiFi Password
(5) WiFi Maximum retries
(192.168.2.136) SNTP server name
(mqtt://192.168.2.136) URI of the MQTT-Broker
(1883) MQTT-Broker Port
(username) Username (if credentials needed)
(password) Password (if credentials needed)
[*] Flash - Light?
(4)   Flash - GPIO
[ ] LED-Strip - Light?
    Camera Frame Size (640x480 (VGA)) --->

[Space/Enter] Toggle/enter  [ESC] Leave menu          [S] Save
[O] Load                  [?] Symbol info          [/] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)

```

Abbildung 4.1: GUI-Konfiguration dank Kconfig-Datei

4.3.4 Weitere Optimierungen

Dank der Nutzung von ESP IDF erhält man umfassende Kontrolle über die Hardware des ESP32-Cam, was es ermöglicht, den Stromverbrauch erheblich zu beeinflussen. Um die Effizienz des Systems weiter zu steigern, wurde eine „sdkconfig.defaults“-Datei eingeführt, die dazu dient, Standardwerte für verschiedene Einstellungen festzulegen.

Zu den bedeutendsten Optimierungen gehört das Ausschalten des zweiten CPU-Kerns des ESP32. Da die Anwendung zu keinem Zeitpunkt eine hohe CPU-Leistung benötigt, kann diese Maßnahme ohne Probleme umgesetzt werden. Zusätzlich wurde die Taktrate des verbleibenden Kerns auf lediglich 80 MHz gesenkt, anstelle der üblichen 240 MHz. Diese Reduzierung hat einen signifikanten Einfluss auf den Stromverbrauch, da der ESP nun weniger Energie benötigt, während er die erforderlichen Aufgaben ausführt.

Darüber hinaus wurde das Bluetooth-Interface deaktiviert, da er in diesem Anwendungsfall nicht verwendet wird. Dies trägt ebenfalls dazu bei, potenziellen Stromverbrauch zu vermeiden und die Effizienz des Systems zu erhöhen. Durch diese Maßnahmen wird sichergestellt, dass „MeterMonitor-ESP“ sowohl leistungsfähig als auch energieeffizient arbeitet.

4.3.5 Probleme

LED-Streifen nicht für DeepSleep geeignet

Während des Testens von „MeterMonitor-ESP“ zeigte sich, dass die verwendeten WS2812B LED-Streifen für diesen Anwendungsfall ungeeignet sind. Nach dem Aufwachen aus dem DeepSleep-Modus war es nicht mehr möglich, die LEDs erfolgreich anzusteuern, selbst wenn die Software korrekt initialisiert und vor dem Schlafen wieder deinitialisiert wurde.

Da diese Art von LED-Streifen auch im ausgeschalteten Zustand durch den Mikrochip pro LED rund 0,5 mA verbrauchen, ist ihr Einsatz in energieeffizienten Anwendungen wie MeterMonitor sowieso ungeeignet. Daher muss auf die Nutzung des eingebauten LED-Flashs des ESP32-Cam zurückgegriffen werden oder es besteht die Möglichkeit, eine externe LED zu verwenden, die softwareseitig ähnlich wie der Flash betrieben werden kann.

USB-Stromversorgung

Beim Versuch, den durchschnittlichen Stromverbrauch des ESP32-Cam mit Powerbanks zu testen, traten in der Praxis Probleme auf. Aufgrund des nun sehr geringen Stromverbrauchs des ESPs im DeepSleep-Modus schalteten sich alle getesteten Powerbanks automatisch ab, da sie die geringe Last nicht als solche anerkannten. Dies stellt ein erhebliches Hindernis dar, da so das System in der Praxis nicht mit Powerbanks betrieben werden kann.

Alternative Optionen, wie das Betreiben des ESPs mit Batteriezellen, wurden ebenfalls in Betracht gezogen. Dieser Ansatz wäre jedoch sehr aufwendig, da ein Battery Management System (BMS) erforderlich wäre, um die Effizienz und vor allem die Sicherheit der Stromversorgung zu gewährleisten.

Die Lösung des Problems fand sich in einem Umstieg vom ESP32-Cam-Board auf das [Seeed Studio XIAO ESP32S3 Sense](#)-Board. Dieses bietet die gleiche Kamera wie der ESP32-Cam, ist jedoch wesentlich kleiner und verfügt dabei über einen integrierten Power-Management-Chip [10]. Dies ermöglicht es, den XIAO ESP32S3 direkt über eine 3,7-V-Lithium-Batterie zu betreiben, die zudem über den USB-C-Anschluss des Boards aufgeladen werden kann.

Der einzige Nachteil des neuen Boards besteht darin, dass es keinen integrierten LED-Flash hat, der für „MeterMonitor“ essenziell ist. Gegen Ende des Projektes wurde jedoch das Betreiben einer externen LED mit diesem Board getestet, was ohne Probleme mit dem Code von „MeterMonitor-ESP“ funktionierte.

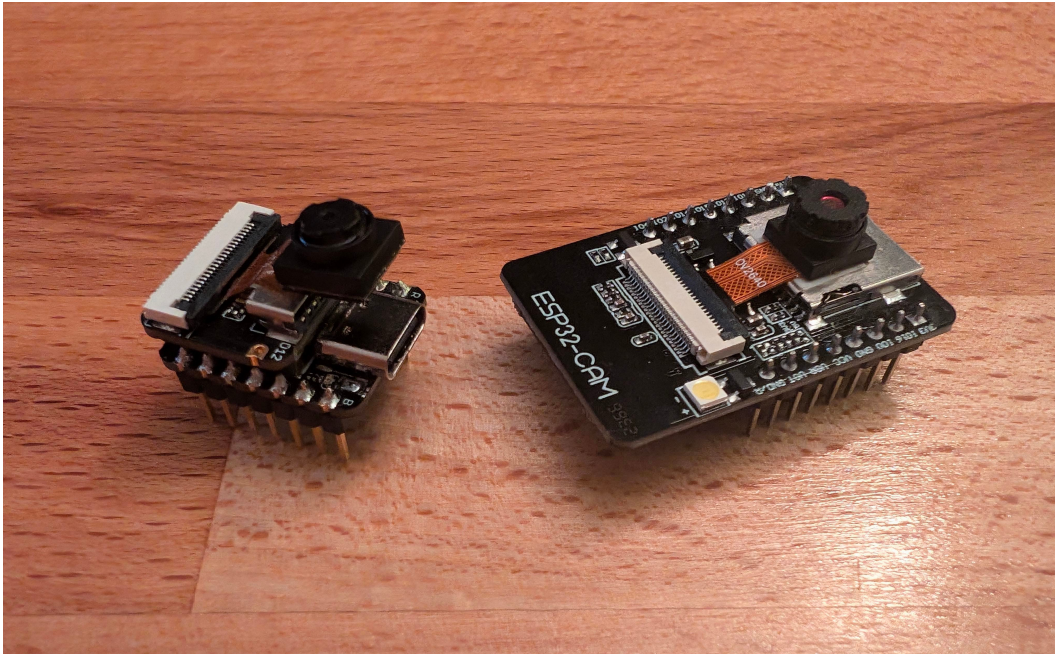


Abbildung 4.2: Größenvergleich des Ai-Thinker ESP32-Cam und des Seeed Studio XIAO ESP32S3 Sense

Aufgrund des Umstiegs auf das neue Board und der Schwierigkeiten, den Stromverbrauch mithilfe von Powerbanks zu testen, konnte der durchschnittliche Stromverbrauch in der Praxis nicht ermittelt werden. Jedoch kann dieser basierend auf den Angaben des Herstellers approximiert werden. Laut Hersteller benötigt der XIAO ESP32S3 Sense beim Streamen von Video per WiFi bei 3.8V der Batterie im Durchschnitt 154mA, und im DeepSleep-Modus mit angesteckter Kamera werden bei Batteriebetrieb im Schnitt 3.00mA verbraucht [11]. Bei einem Messintervall von 5 Minuten und einer aktiven Phase von 20 Sekunden ergibt sich ein durchschnittlicher Stromverbrauch von etwa 13 mA. Ein 5000mAh-Akku würde somit bis zu 16 Tage halten, was eine signifikante Verbesserung zu den 48 Stunden beim Testen von „AI-on-the-edge“ darstellt. Außerdem ist zu beachten, dass dies nur eine sehr grobe Schätzung ist und der tatsächliche Stromverbrauch vor allem in der aktiven Phase geringer ausfallen sollte. Die Dauer der aktiven Phase wurde mit 20 Sekunden absichtlich hoch angesetzt, um eine pessimistische Schätzung des Stromverbrauchs zu liefern. In der Praxis lag die Dauer der aktiven Phase eher bei etwa 10 Sekunden, was den Stromverbrauch weiter reduzieren würde.

4.4 MeterMonitor-Classifler

Wie in Abschnitt 4.2 beschrieben, wurde ein großer Teil der Aufgaben des ESP32-Cams auf einen zentralen Server ausgelagert. Für diese Aufgaben ist „MeterMonitor-Classifler“ verantwortlich.

Der in Python entwickelte Code läuft durchgängig auf einem Server und empfängt sowie verarbeitet die von „MeterMonitor-ESP“ über den MQTT-Broker gesendeten Nachrichten. Python wurde als Programmiersprache gewählt, da es eine schnelle Entwicklung ermöglicht und man in diesem Fall nicht auf eine hohe Energieeffizienz angewiesen ist. Zudem ist Python besonders gut für OCR (Optical Character Recognition) und maschinelles Lernen geeignet, was MeterMonitor zugutekommt, da die Bildverarbeitung und Erkennung des Wasserzählerstands zu den Kernaufgaben des Systems gehören.

Ein weiterer Vorteil von Python ist seine hohe Abstraktionsebene, die es einfach macht, mit MQTT zu arbeiten. So konnte problemlos auf den Topic „MeterMonitor/#“ subscribed werden, wobei das #-Zeichen bedeutet, dass auch über alle Untertopics empfangen werden soll. Dies ist wichtig, da jede ESP32-Cam-Instanz ihre Nachrichten auf ein eigenes Untertopic entsprechend dem Namen des Wasserzählers veröffentlicht. „MeterMonitor-Classifler“ ist dadurch für den gleichzeitigen Betrieb mit mehreren Instanzen vorbereitet und muss zum Einpflegen neuer Instanzen nicht manuell angepasst werden.

Da das Erkennen der Wasserzählerstände potenziell zeit- und rechenintensiv ist, wurde die Verarbeitung parallelisiert, um sicherzustellen, dass während der Auswertung eines Bildes weitere Nachrichten und Bilder nicht verpasst werden. Dies wurde durch die Nutzung eines Multiprocessing-Pools umgesetzt, an den die OCR-Aufgabe übergeben wird, sobald eine Nachricht empfangen und das Bild gespeichert wurde.

Sobald eine Nachricht eingeht, werden alle relevanten Informationen aus dem JSON extrahiert und in die EXIF-Daten des gespeicherten JPEG-Bildes eingetragen. Da nicht alle Informationen ein eigenes EXIF-Feld haben, wird das UserComment-Feld genutzt, um die restlichen Daten aus der JSON-Nachricht in das Bild zu kodieren.

Obwohl im Rahmen dieser Arbeit das eigentliche Erkennen des Zählerstandes aufgrund von Zeitmangel nicht eingebaut wurde, ist das System vollständig vorbereitet, um diese Funktionalität in einem weiteren Verlauf hinzuzufügen.

5 Zusammenfassung

5.1 Fazit

Im Rahmen dieses Masterprojekts wurde ein bedeutender Schritt in Richtung eines Smart Campus unternommen. Ziel war es, eine Lösung zur Digitalisierung der zahlreichen Wasserzähler der Hochschule RheinMain zu entwickeln. Zu Beginn wurde das Open-Source-Projekt „**AI-on-the-edge**“ evaluiert und mit einer eigens entwickelten, parametrisierten 3D-druckbaren **Wasserzähler-Halterung** in kleinem Rahmen getestet. Die Ergebnisse zeigten jedoch, dass „AI-on-the-edge“ für den großflächigen Einsatz auf dem Campus aufgrund des hohen Stromverbrauchs und vorhandener Sicherheitsbedenken nicht geeignet ist.

Aus diesen Erkenntnissen heraus wurde **MeterMonitor** entwickelt – ein System, das speziell darauf ausgelegt ist, die Wasserzähler stromsparend zu digitalisieren. Durch den Einsatz des Ai-Thinker ESP32-Cam [4] und später des [Seeed Studio XIAO ESP32S3 Sense](#) [10] konnte eine Lösung entwickelt werden, die es ermöglicht, regelmäßig Bilder der Wasserzählerstände zu erfassen und diese per MQTT an einen zentralen Server zu übertragen. Dort erfolgt die weitere Digitalisierung campusweit mithilfe von OCR (Optical Character Recognition).

Trotz der im Verlauf aufgetretenen Herausforderungen und Probleme, wie der Frage der Energieversorgung und der Hardwareauswahl, konnte das Konzept im Prototyp erfolgreich umgesetzt und validiert werden. MeterMonitor bietet somit eine vielversprechende Grundlage für den weiteren Ausbau und die flächendeckende Implementierung einer intelligenten Wasserzählerüberwachung an der Hochschule.

5.2 Ausblick

Implementierung der Zählerstandserkennung mittels OCR

Wie bereits in Abschnitt 4.4 beschrieben, konnte die eigentliche Digitalisierung der Zählerstände in „MeterMonitor“ aufgrund von Zeitmangel nicht vollständig implementiert werden. Für diese Aufgabe könnten die bereits trainierten TensorFlow Lite-Modelle des „AI-on-the-edge“-Projekts genutzt werden. Da die Bildverarbeitung jedoch nicht mehr direkt auf dem ESP32-Cam, sondern auf einem leistungstärkeren Server stattfindet, eröffnet sich die Möglichkeit,

weitaus komplexere und präzisere Modelle zur Erkennung der Zählerstände einzusetzen.

„MeterMonitor-Classifer“ wurde bereits so vorbereitet, dass eine Methode zur Integration der Zählerstandserkennung mittels OCR leicht implementiert werden kann. Diese läuft parallel zum restlichen Datenfluss und ermöglicht eine flexible und skalierbare Erweiterung des Systems, sodass in zukünftigen Projektschritten die Digitalisierung der Wasserzähler effizient umgesetzt werden kann.

Finalisierung des Umstiegs auf das „Seeed XIAO ESP32S3 Sense“-Board

Wie bereits in Abschnitt 4.3.5 beschrieben, wurde aufgrund der Problematik des Betriebs des ESP32-Cams mit Powerbanks der Umstieg auf das Seeed Studio XIAO ESP32S3 Sense [10] vorgenommen. Dieses Board verfügt über einen integrierten Power-Management-Chip, wodurch ein direkter Betrieb per Akku ermöglicht wird.

Der bestehende Code von „MeterMonitor-ESP“ wurde bereits angepasst, um auf dem neuen Board zu laufen. Tests haben gezeigt, dass der Code nun ohne nennenswerte Unterschiede im Vergleich zum Betrieb auf dem ESP32-Cam funktioniert.

Um jedoch ein vollständiges Deployment des Systems in Betracht zu ziehen, ist es notwendig, das Gehäuse an die kompaktere Platine anzupassen. Die Reduzierung der Platinenfläche des Seeed XIAO ESP32S3 Sense erfordert eine Überarbeitung des Gehäuses. Gleichzeitig muss darauf geachtet werden, dass eine Kompatibilität mit dem bestehenden ESP32-Cam-Board gewahrt bleibt. Daher ist es notwendig, ein neues Platinen-Gehäuse zu entwickeln, das in die bereits vorhandene „Kamera-Halterung“ des in diesem Projekt entwickelten Gehäuses passt. Dies ermöglicht nicht nur eine nahtlose Integration des neuen Boards, sondern sorgt auch dafür, dass die Flexibilität und Modularität des Systems beibehalten bleibt.

Des Weiteren sollte in dieses Gehäuse eine Halterung für die zwingend erforderliche externe Flash-LED eingeplant werden, und es muss ein Weg gefunden werden, um die Reflexionen dieser im Wasserzähler zu minimieren. Darüber hinaus sollte mit dem neuen Board der durchschnittliche Stromverbrauch von „MeterMonitor-ESP“ mit dem Akku getestet werden, um die Effizienz und Betriebsdauer des Systems genau zu bestimmen.

Abbildungsverzeichnis

2.1	Aufbau der offiziellen Halterung von „AI-on-the-edge“ [2]	3
2.2	Entwickelter Schnappverschluss zum Verbinden der drei Hauptkomponenten	7
2.3	Beispiel des entwickelten Gehäuses für größere Wasserzähler (Bild wurde mithilfe von Ray-Tracing in Fusion 360 gerendert) .	8
2.4	Geslicte Buildplate mit Stützen für die Kamerahalterung und den USB-Port	9
3.1	Schematischer Arbeitsablauf von „AI-on-the-edge“	11
3.2	Flashen des ESP32-Cam mithilfe eines USB-Serial-Adapters [5] .	12
3.3	Links: ESP32-CAM / OV2640 lens wrench (von: mkarliner) [6] Rechts: ESP32-CAM Focus tool (von: AskePetter) [7]	13
3.4	Weboberfläche von „AI-on-the-edge“ im Normalbetrieb	14
3.5	Testaufbau von „AI-on-the-edge“ mit externer Antenne	15
3.6	oben: Eingebauter Flash (links ohne, rechts mit Diffusor) unten: Externer LED-Streifen (links ohne, rechts mit Diffusor)	17
4.1	GUI-Konfiguration dank Kconfig-Datei	25
4.2	Größenvergleich des Ai-Thinker ESP32-Cam und des Seeed Studio XIAO ESP32S3 Sense	27

Literaturverzeichnis

- [1] jomjol. Github - jomjol/ai-on-the-edge-device: Easy to use device for connecting “old” measuring units (water, power, gas, ...) to the digital world. <https://github.com/jomjol/AI-on-the-edge-device>, abgerufen am 19. September 2024.
- [2] jomjol. water meter / wasserzähler - ai-on-the-edge. <https://www.thingiverse.com/thing:4573481>, abgerufen am 20. September 2024.
- [3] Autodesk Inc. Autodesk for Students - Product Page. <https://www.autodesk.de/education/edu-software/overview>, abgerufen am 21. September 2024.
- [4] Ai-Thinker. ESP32-CAM camera development board. <https://docs.ai-thinker.com/en/esp32-cam>, abgerufen am 23. September 2024.
- [5] raphaelbs (Raphael Brandão). Wiring-with-usb-ttl.png. <https://github.com/raphaelbs/esp32-cam-ai-thinker/blob/master/assets/Wiring-with-usb-ttl.png>, abgerufen am 23. September 2024.
- [6] mkarliner (Michael Karliner). ESP32-CAM / OV2640 lens wrench. <https://www.thingiverse.com/thing:4783689>, abgerufen am 23. September 2024.
- [7] AskePetter (Petter Nordgren). ESP32-CAM Focus tool. <https://www.thingiverse.com/thing:4844956>, abgerufen am 23. September 2024.
- [8] Espressif Systems. ESP32 Camera Driver. <https://github.com/espressif/esp32-camera>, abgerufen am 25. September 2024.
- [9] okhsunrog (Danila Gornushko). A light and simple ESP-IDF lib for WS2812B/WS2815 led strips. https://github.com/okhsunrog/esp_ws28xx, abgerufen am 25. September 2024.
- [10] Inc Seeed Studio. Getting Started with Seeed Studio XIAO ESP32S3 (Sense). https://wiki.seeedstudio.com/xiao_esp32s3_getting_started/#battery-usage, abgerufen am 25. September 2024.

- [11] Inc Seeed Studio. Getting Started with Seeed Studio XIAO ESP32S3 (Sense). https://wiki.seeedstudio.com/xiao_esp32s3_getting_started/#specification, abgerufen am 25. September 2024.